

PROJET LONG IDE7

Mini IDE Java

avec analyse UML, génération de code et autocomplétion

Rapport technique global

Auteurs :

Mouad Mardy

ESMAHI Salah eddine

Ziani Mohamed Amine

Jaouad Ait Nasser

Yahya El Morhder

Yasser Errerhaoui

Ahmed El Fanaoui

Saad Lefrais

Année universitaire 2025–2026

Table des matières

1	Introduction générale	3
2	Analyse de code, modélisation UML et génération Java	4
2.1	Contexte et objectifs	4
2.2	Répartition des contributions	4
2.2.1	Travail réalisé par Mouad Mardy	4
2.2.2	Travail réalisé par ESMAHI Salah eddine	4
2.3	Conception du modèle UML	5
2.3.1	Classe JavaClass	5
2.3.2	Classe JavaAttribut	5
2.3.3	Classe JavaMethode	5
2.3.4	Classe JavaParametre	5
2.3.5	Classe JavaRelation	5
2.3.6	Énumération Visibility	5
2.4	Lecture du fichier Java	6
2.5	Analyse du code source	6
2.6	Extraction des éléments du code	6
2.7	Génération de code Java	6
2.8	Affichage du diagramme UML	7
2.9	Bibliothèques utilisées	7
2.10	Validation	7
3	Interface graphique JavaFX	8
3.1	Contexte	8
3.2	Travail réalisé par Ziani Mohamed Amine et Jaouad Ait Nasser	8
3.3	Fonctionnalités d'édition	8
3.4	Aspects techniques	9
3.4.1	Vue principale : MainView.java	9
3.4.2	Contrôleur principal : MainController.java	9
3.4.3	Ouverture de projet	9
3.4.4	Rendu de l'explorateur	9
3.5	Validation	9
3.6	Limites et suites possibles	9
4	Module d'autocomplétion	11
4.1	Contexte	11
4.2	Auteurs	11
4.3	Architecture du module	11
4.4	Prérequis	11

4.5	Principe de fonctionnement	11
4.6	Utilisation de KeywordExtractor	12
4.6.1	Tokenisation	12
4.7	Utilisation de Completer	12
4.7.1	Exécution	12
4.7.2	Exemple de session	12
4.8	Tests	13
4.8.1	TrieTest	13
4.8.2	KeywordExtractorTest	13
4.9	Perspectives	13
5	Module Debug & Exécution	14
5.1	Contexte	14
5.2	Auteurs	14
5.3	Travail réalisé	14
5.4	Principe de fonctionnement	14
5.5	Architecture du module	15
5.6	Validation	15
5.7	Limites	15
5.8	Perspectives	15
6	Bilan global, limites et perspectives	17
6.1	Bilan global	17
6.2	Limites globales	17
6.3	Perspectives d'évolution	17
7	Conclusion	18

Chapitre 1

Introduction générale

Le projet IDE7 a pour objectif la conception et la réalisation d'un mini environnement de développement intégré pour le langage Java. Le système développé vise à fournir un ensemble cohérent d'outils permettant d'améliorer la compréhension, l'édition et l'assistance à la programmation.

Dans cette première phase, plusieurs modules complémentaires ont été conçus et réalisés :

- un module d'analyse de code Java et de génération de diagrammes UML ;
- un module de génération automatique de code Java à partir d'un modèle UML ;
- une interface graphique JavaFX structurée selon l'architecture MVC ;
- un module d'autocomplétion fondé sur l'extraction de mots-clés et une structure de type Trie ;
- un module de debug et d'exécution permettant la compilation et l'exécution de programmes Java.

L'ensemble constitue une base fonctionnelle solide pour un IDE Java modulaire, extensible et évolutif.

Chapitre 2

Analyse de code, modélisation UML et génération Java

2.1 Contexte et objectifs

Ce module ajoute deux fonctionnalités majeures au projet IDE7 :

- la génération automatique d'un diagramme UML à partir d'un fichier Java fourni par l'utilisateur ;
- la génération automatique de code Java à partir d'un modèle UML structuré en mémoire.

L'objectif principal est d'assurer un lien bidirectionnel entre la conception et l'implémentation, tout en facilitant la compréhension de la structure d'un programme.

Lors de cette première itération, le travail réalisé a porté sur :

- l'analyse d'un fichier Java fourni par l'utilisateur ;
- l'extraction des éléments essentiels : classe, attributs, méthodes et paramètres ;
- la production d'une représentation UML textuelle dans le terminal ;
- la génération de code Java valide à partir d'un modèle UML défini en mémoire.

2.2 Répartition des contributions

2.2.1 Travail réalisé par Mouad Mardy

Les principales contributions sont les suivantes :

- mise en place du modèle UML interne ;
- développement du module d'analyse du code source Java ;
- mise en place d'un système d'affichage textuel du diagramme UML dans le terminal ;
- ajout de mécanismes simples de gestion des erreurs ;
- réalisation de tests sur des fichiers Java simples.

2.2.2 Travail réalisé par ESMAHI Salah eddine

Les principales contributions sont les suivantes :

- extension du modèle UML avec les champs `isAbstract` et `isInterface` ;
- implémentation du générateur de code Java dans la classe `JavaCodeGenerator` ;
- génération automatique des constructeurs ;
- génération automatique des getters et setters ;
- création d'un fichier de test indépendant `TestCodeGenerator.java`.

2.3 Conception du modèle UML

La représentation interne des éléments UML repose sur plusieurs classes Java.

2.3.1 Classe `JavaClass`

La classe `JavaClass` représente une classe Java dans le modèle UML. Elle contient :

- le nom de la classe ;
- une liste d'attributs ;
- une liste de méthodes ;
- les indicateurs `isAbstract` et `isInterface`.

2.3.2 Classe `JavaAttribut`

La classe `JavaAttribut` représente un attribut et stocke :

- le nom ;
- le type ;
- la visibilité.

2.3.3 Classe `JavaMethode`

La classe `JavaMethode` représente une méthode ou un constructeur. Elle contient :

- le nom de la méthode ;
- le type de retour ;
- la visibilité ;
- une liste de paramètres.

2.3.4 Classe `JavaParametre`

La classe `JavaParametre` représente un paramètre de méthode avec :

- son nom ;
- son type.

2.3.5 Classe `JavaRelation`

La classe `JavaRelation` a été prévue pour représenter les relations entre classes. Bien qu'elle ne soit pas encore pleinement exploitée, elle prépare les extensions futures.

2.3.6 Énumération `Visibility`

L'énumération `Visibility` normalise les niveaux de visibilité avec les valeurs :

- `PUBLIC`

- `PRIVATE`
- `PROTECTED`

2.4 Lecture du fichier Java

La lecture du fichier fourni par l'utilisateur est réalisée dans la classe `Main`. Le programme reçoit en argument le nom du fichier à analyser. Si aucun fichier n'est fourni, un message d'usage est affiché afin d'indiquer la manière correcte d'exécuter le programme.

2.5 Analyse du code source

L'analyse est prise en charge par la classe `JavaParser`. La méthode principale `parse(String source)` orchestre le processus d'analyse :

- recherche du nom de la classe ;
- création de l'objet `JavaClass` ;
- ajout des attributs détectés ;
- ajout des méthodes et constructeurs identifiés.

Le parseur repose sur des expressions régulières pour identifier les structures essentielles du langage Java.

2.6 Extraction des éléments du code

Le parseur détecte :

- les attributs par repérage des lignes contenant visibilité, type et nom suivis d'un point-virgule ;
- les méthodes et constructeurs à partir de leur signature ;
- les niveaux de visibilité, qui sont convertis en valeurs de l'énumération `Visibility`.

Cette approche reste simple mais permet de constituer une première base fonctionnelle.

2.7 Génération de code Java

La génération est assurée par la classe `JavaCodeGenerator`. À partir d'un objet `JavaClass`, le générateur reconstruit le code Java correspondant.

Les étapes de génération sont les suivantes :

- déclaration de la classe, de la classe abstraite ou de l'interface ;
- génération des attributs ;
- génération des constructeurs ;
- génération des getters et setters ;
- génération des méthodes.

Pour chaque attribut de visibilité `private`, le générateur produit automatiquement :

- un getter ;
- un setter ;
- un constructeur vide ;
- un constructeur complet initialisant les attributs via `this`.

2.8 Affichage du diagramme UML

L'objet `JavaClass` complet est transmis à la classe `UMLPrinter`. La méthode `print(JavaClass javaClass)` affiche un diagramme UML textuel dans le terminal.

L'affichage suit les conventions UML standards :

- `+` pour `public`;
- `-` pour `private`;
- `#` pour `protected`.

2.9 Bibliothèques utilisées

Les principales bibliothèques utilisées sont :

- `java.util` pour les listes et collections;
- `java.util.regex` pour l'analyse syntaxique;
- `java.nio.file` pour la lecture des fichiers;
- `java.util.stream` pour certaines opérations de filtrage et de construction.

2.10 Validation

La compilation et l'exécution ont été réalisées à l'aide des commandes suivantes :

```
1 javac *.java
2 java Main Point.java
```

Lors de l'exécution, le programme lit le fichier Java et affiche une représentation UML textuelle dans le terminal. Le générateur de code a été validé séparément par le fichier `TestCodeGenerator.java`.

Chapitre 3

Interface graphique JavaFX

3.1 Contexte

Cette partie concerne l'interface d'un mini-IDE JavaFX structuré selon l'architecture MVC avec les couches **views**, **controllers** et **models**. Une base fonctionnelle existait déjà pour la création et l'ouverture de projet ainsi que pour un éditeur de texte simple. Cette base a ensuite été enrichie afin d'améliorer l'ergonomie et la productivité.

3.2 Travail réalisé par Ziani Mohamed Amine et Jaouad Ait Nasser

Les améliorations apportées à l'interface sont les suivantes :

- mise en place et consolidation de l'interface principale ;
- gestion des projets : création, ouverture et retour vers l'accueil ;
- création de fichiers, de classes Java et de packages ;
- navigation dans l'arborescence et ouverture des fichiers ;
- sauvegarde manuelle et auto-sauvegarde ;
- enrichissement de l'éditeur ;
- ajout d'une barre de menus type IDE ;
- uniformisation linguistique en français ;
- correction de bugs d'expérience utilisateur.

3.3 Fonctionnalités d'édition

Les fonctionnalités intégrées à l'éditeur incluent :

- numéros de ligne ;
- recherche et remplacement ;
- annuler, rétablir, couper, copier, coller ;
- sélection intégrale ;
- barre d'outils avec icônes.

3.4 Aspects techniques

3.4.1 Vue principale : `MainView.java`

Cette vue inclut :

- une `MenuBar` avec différentes rubriques ;
- une barre d’outils pour l’édition ;
- l’intégration de la couche de coloration syntaxique ;
- la synchronisation du défilement et des numéros de ligne.

3.4.2 Contrôleur principal : `MainController.java`

Le contrôleur gère :

- le lien entre les actions de menu et la logique métier ;
- l’ouverture des fichiers au clic ;
- la gestion de l’état *dirty* ;
- la recherche et le remplacement.

3.4.3 Ouverture de projet

Les classes `OpenProjectView.java` et `OpenProjectController.java` permettent :

- une vue adaptée aux petits écrans ;
- l’ajout d’un bouton « Nouveau projet » depuis la vue d’ouverture.

3.4.4 Rendu de l’explorateur

L’explorateur de projet intègre des icônes distinctes pour les dossiers et les fichiers afin d’améliorer la lisibilité de l’arborescence.

3.5 Validation

La validation a été réalisée avec les commandes suivantes :

```
1 mvn compile
2 mvn javafx:run
```

Des vérifications manuelles ont également été effectuées sur :

- la création et l’édition de fichiers `.java` et `.txt` ;
- la visibilité correcte du texte à la frappe ;
- la cohérence des menus et commandes ;
- l’ergonomie sur fenêtre standard.

3.6 Limites et suites possibles

Les principales limites identifiées sont :

- une coloration syntaxique encore basique ;
- l’absence de raccourcis clavier avancés ;
- un ensemble de préférences utilisateur non persistantes.

Parmi les évolutions possibles :

- thèmes clair et sombre ;
- onglets multi-fichiers ;
- minimap et pliage de code ;
- préférences d'éditeur persistantes.

Chapitre 4

Module d'autocomplétion

4.1 Contexte

Le module d'autocomplétion a pour objectif d'extraire les mots-clés d'un fichier source Java, de les indexer dans une structure de données efficace, puis de proposer à l'utilisateur une liste de suggestions classées par fréquence à partir d'un préfixe donné.

4.2 Auteurs

Ce module a été réalisé par Yahya EL MORHDER et Yasser ERRERHAOUI.

4.3 Architecture du module

Le module est composé des classes principales suivantes :

- **Completer** : point d'entrée interactif ;
- **KeywordExtractor** : extraction et indexation des mots-clés ;
- **Trie** : structure de données de type arbre préfixe ;
- **TrieNode** : nœud interne du Trie ;
- **KeywordExtractorTest** et **TrieTest** : suites de tests JUnit 4.

4.4 Prérequis

L'utilisation du module nécessite :

- Java JDK version 11 ou supérieure ;
- JUnit 4 dans le classpath ;
- l'accès aux commandes `java` et `javac` ;
- les fichiers sources présents dans le paquetage concerné ;
- un fichier texte ou Java à analyser.

4.5 Principe de fonctionnement

Le module suit une architecture en couches :

- extraction des mots-clés depuis un fichier ;
- alimentation d'un dictionnaire de fréquences ;
- insertion dans un Trie ;
- génération de suggestions à partir d'un préfixe.

4.6 Utilisation de KeywordExtractor

La classe `KeywordExtractor` analyse un fichier et remplit deux structures complémentaires :

- un `HashMap<String, Integer>` accessible via `getKeywordsDictionary()` ;
- un `Trie` accessible via `getKeywordsTrie()`.

4.6.1 Tokenisation

Chaque ligne du fichier est découpée à l'aide d'une expression régulière destinée à éliminer la ponctuation et plusieurs opérateurs Java courants :

```
1 [ ,\\.\\s\\;\\:\\<\\>\\{\\}\\[\\]\\(\\)\\=\\-\\+\\/\\!\\@\\&\\|\\\"'"]
```

Seuls les mots de longueur strictement supérieure à 2, non vides et sans retour à la ligne sont conservés.

4.7 Utilisation de Completer

La classe `Completer` constitue le point d'entrée interactif. Son exécution suit les étapes suivantes :

1. demander à l'utilisateur la taille maximale de la liste de suggestions ;
2. demander le chemin du fichier à analyser ;
3. lancer l'extraction ;
4. entrer dans une boucle interactive de saisie de préfixe ;
5. afficher les suggestions triées par fréquence décroissante.

4.7.1 Exécution

```
1 java -cp bin yer1420.Completer
```

4.7.2 Exemple de session

```
1 Enter suggestion list size : 5
2 Enter filepath :
3 ./src/yer1420/Trie.java
4 Scanning file ...
5 Type word to complete :
6 Tri
7 Searching for completions ...
8 List of suggestions :
9 TrieNode
```

```
10 Trie
11 TrieTest
```

4.8 Tests

4.8.1 TrieTest

```
1 java -cp bin:junit-4.x.jar org.junit.runner.JUnitCore yer1420.
  TrieTest
```

4.8.2 KeywordExtractorTest

La classe `KeywordExtractorTest` contient un cas de test appelant `extractFromFile` sur `Trie.java` avec une vérification visuelle des mots extraits.

```
1 java -cp bin:junit-4.x.jar org.junit.runner.JUnitCore yer1420.
  KeywordExtractorTest
```

4.9 Perspectives

Les évolutions envisagées pour les itérations suivantes sont :

- implémentation plus complète du Trie ;
- meilleure gestion des nouveaux mots dans un fichier modifié ;
- intégration d'éléments du JDK dans les suggestions ;
- ajout d'une interface graphique avec console d'autocomplétion intégrée à l'IDE.

Chapitre 5

Module Debug & Exécution

5.1 Contexte

Ce module permet la compilation et l'exécution de programmes Java en s'appuyant sur les outils système (`javac`, `java`). Il assure également la capture des résultats d'exécution et leur structuration.

5.2 Auteurs

Ce module a été réalisé par **Ahmed El Fanaoui** et **Saad Lefrais**.

5.3 Travail réalisé

- Implémentation du service de compilation via `CompilerService`
- Implémentation du service d'exécution via `ExecutionService`
- Mise en place du chaînage `compileAndRun`
- Encapsulation des résultats dans `ExecutionResult`
- Mise en place d'une structure de suivi mémoire `MemorySnapshot`
- Validation fonctionnelle via la classe `MonTest`

5.4 Principe de fonctionnement

La compilation est lancée via `ProcessBuilder` en appelant `javac`. L'exécution est ensuite réalisée via la commande `java -cp bin`.

En cas d'échec de compilation, l'exécution est automatiquement bloquée. Tous les résultats sont regroupés dans un objet `ExecutionResult`.

5.5 Architecture du module

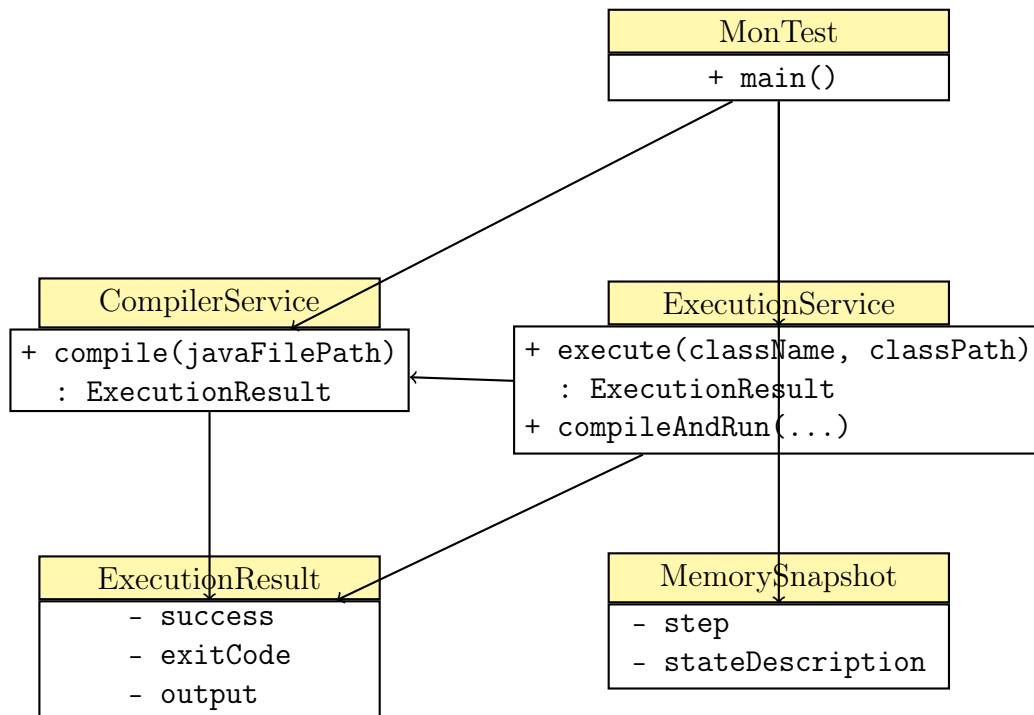


FIGURE 5.1 – UML du module Debug & Exécution

5.6 Validation

Les tests ont été réalisés à l’aide de la classe `MonTest`. Ils couvrent :

- la compilation d’un programme Java ;
- son exécution ;
- le chaînage compilation + exécution ;
- la capture des sorties.

Les résultats obtenus confirment le bon fonctionnement global du module.

5.7 Limites

Les limitations actuelles sont les suivantes :

- absence de parsing structuré des erreurs de compilation ;
- absence de débogage pas-à-pas ;
- gestion simplifiée du classpath.

5.8 Perspectives

Les améliorations envisagées sont :

- parsing détaillé des erreurs de compilation ;
- intégration d’une console graphique dans l’IDE ;
- ajout d’un débogueur pas-à-pas ;

- enrichissement de la structure `MemorySnapshot`.

Chapitre 6

Bilan global, limites et perspectives

6.1 Bilan global

Le projet IDE7 réunit plusieurs briques logicielles complémentaires :

- analyse statique de code Java ;
- modélisation UML ;
- génération automatique de code ;
- interface graphique de type IDE ;
- autocomplétion fondée sur une structure de données efficace ;
- compilation et exécution de programmes Java.

Cette première version constitue une base sérieuse pour le développement d'un environnement de programmation plus complet.

6.2 Limites globales

Les limites actuelles du projet sont les suivantes :

- analyse Java reposant sur des expressions régulières ;
- gestion limitée des relations UML ;
- autocomplétion restreinte au contenu analysé ;
- interface encore simplifiée par rapport à un IDE complet ;
- fonctionnalités de debug encore limitées.

6.3 Perspectives d'évolution

Les perspectives principales sont :

- utilisation d'un parseur syntaxique plus robuste ;
- gestion complète des relations UML ;
- export de fichiers `.java` sur disque ;
- visualisation graphique des diagrammes UML ;
- autocomplétion enrichie par le JDK et le contexte du projet ;
- support multi-fichiers et personnalisation de l'interface ;
- ajout d'un débogage avancé et d'une console intégrée.

Chapitre 7

Conclusion

Le projet IDE7 met en évidence une démarche de conception modulaire où plusieurs composants collaborent pour offrir une première version cohérente d'un mini IDE Java.

L'analyse UML, la génération de code, l'interface graphique JavaFX, l'autocomplétion ainsi que le module de debug et d'exécution constituent des fonctionnalités complémentaires qui renforcent à la fois la compréhension du code, l'assistance au développement et l'expérience utilisateur.

Cette première itération fournit ainsi une base fonctionnelle crédible et extensible pour les développements futurs.